

INSTITUT NATIONAL DES SCIENCES
APPLIQUÉES DE ROUEN

INSA DE ROUEN



GM3, PROJET MIPP

Labyrinthe

Auteurs :

Manh Hung NGUYEN

mnguy00@insa-rouen.fr

Tan Minh Duy NGO

tn00@insa-rouen.fr

Enseignant :

Julien SAUNIER

julien.saunier@insa-rouen.fr

16 Octobre 2025

Résumé

Ce projet a pour objectif de développer une application capable de générer et d'explorer des labyrinthes à l'aide de l'algorithme Growing Tree. Implémenté en langage C, le programme propose plusieurs méthodes de tirage permettant de comparer l'impact de chacune sur la forme et la complexité du labyrinthe obtenu.

Le code a été conçu de manière modulaire, avec une séparation claire entre la génération, l'affichage et les fonctions de gestion. En plus du mode d'affichage ASCII, une version interactive utilisant la bibliothèque *ncurses* permet à l'utilisateur de se déplacer à l'intérieur du labyrinthe et de chercher la sortie.

La réalisation de ce projet nous a permis d'approfondir nos compétences en algorithmique et en programmation en C, notamment sur la gestion des pointeurs et de la mémoire dynamique. Malgré les difficultés initiales liées à ce langage, nous avons acquis une meilleure maîtrise de la logique du C et du développement modulaire.

Keywords: *Labyrinthe, Growing Tree, langage C, pointeurs, gestion de mémoire*

Table des matières

1	Introduction	2
2	Analyse et modélisation	3
3	Implémentation	5
4	Jeux d’essais et résultats	7
4.1	Méthode theMRC (Most Recent Cell)	7
4.2	Méthode randomList	8
4.3	Méthode lastNList	8
4.4	Méthode lastOrRand	9
4.5	Comparaison	10
4.6	Mode interactif	10
5	Synthèse critique	11
6	Conclusion	12

Chapitre 1

Introduction

Dans le cadre du module de projet MIPP, nous avons réalisé une application capable de générer et d'explorer des labyrinthes. L'objectif principal de ce travail était de comprendre et d'implémenter un algorithme de génération procédurale, en particulier l'algorithme du *Growing Tree*, tout en assurant une modélisation claire et un affichage fonctionnel.

Le principe de cet algorithme repose sur la création progressive d'un labyrinthe à partir d'une cellule initiale, puis sur l'ouverture de passages selon différentes stratégies de sélection. Ces stratégies, appelées méthodes de tirage, influencent fortement la forme et la complexité du labyrinthe généré.

Dans notre projet, nous avons choisi d'implémenter toutes les méthodes proposées - *most recent*, *random*, *last or random* et *last n* - afin de comparer leurs effets visuels et structurels. Cette approche nous a permis de mieux comprendre le rôle du tirage dans la génération et d'observer la diversité des labyrinthes produits.

En plus de la génération, notre programme permet le déplacement d'un personnage à l'intérieur du labyrinthe. L'utilisateur peut ainsi interagir directement avec labyrinthe généré, ce qui rend le projet démonstratif.

Ce rapport présente dans un premiers temps l'analyse et la modélisation du problème, puis les choix techniques faits lors de l'implémentation. Enfin, nous exposerons les résultats obtenus, leur interprétation et une synthèse critique du travail réalisé.

Chapitre 2

Analyse et modélisation

Avant de commencer l'implémentation, nous avons analysé le fonctionnement de l'algorithme *Growing Tree* afin de comprendre comment modéliser les éléments essentiels du labyrinthe. Le but était de définir des structures de données simples mais efficaces pour représenter les cellules, les murs et la liste des cellules actives.

Le labyrinthe est représenté sous la forme d'une grille de cellules. Chaque cellule possède quatre murs (nord, est, sud, ouest) et un état visité ou non. L'algorithme construit progressivement un labyrinthe parfait. Pour cela, on maintient une liste dynamique de cellules actives dans laquelle on choisit une cellule selon une méthode de tirage donnée. A partir de cette cellule, on ouvre un passage vers un voisin non visité et on continue jusqu'à ce que toutes les cellules aient été visitées.

La modélisation a été réalisée sous forme de plusieurs structures :

- **Cell** : représente une cellule du labyrinthe, avec ses coordonnées (x, y) , son état visité et un tableau de booléens indiquant la présence des murs.
- **Maze** : matrice à deux dimensions contenant l'ensemble des cellules.
- **ListCell** : un tableau de Cell et le nombre d'éléments dans la liste.
- **ArrayOfBool** et **Direction** : types axiliaires permettant de gérer plus facilement les directions et les voisins d'une cellule.

L'algorithme *Growing Tree* commence par choisir une cellule initial que l'on marque comme visitée. Ensuite, on répète les étapes suivantes :

1. Sélectionner une cellule active dans la liste selon une méthode de tirage.
2. Chercher parmi ses voisins une cellule non visitée.
3. Si un voisin libre existe, on ouvre un passage entre les deux cellules en

supprimant les murs correspondants et on ajoute la nouvelle cellule à la liste.

4. Si la cellule n'a plus de voisins non visités, elle est retirée de la liste.

L'algorithme se termine quand la liste est vide, ce qui signifie que toutes les cellules ont été explorées.

Voici notre analyse descendant :

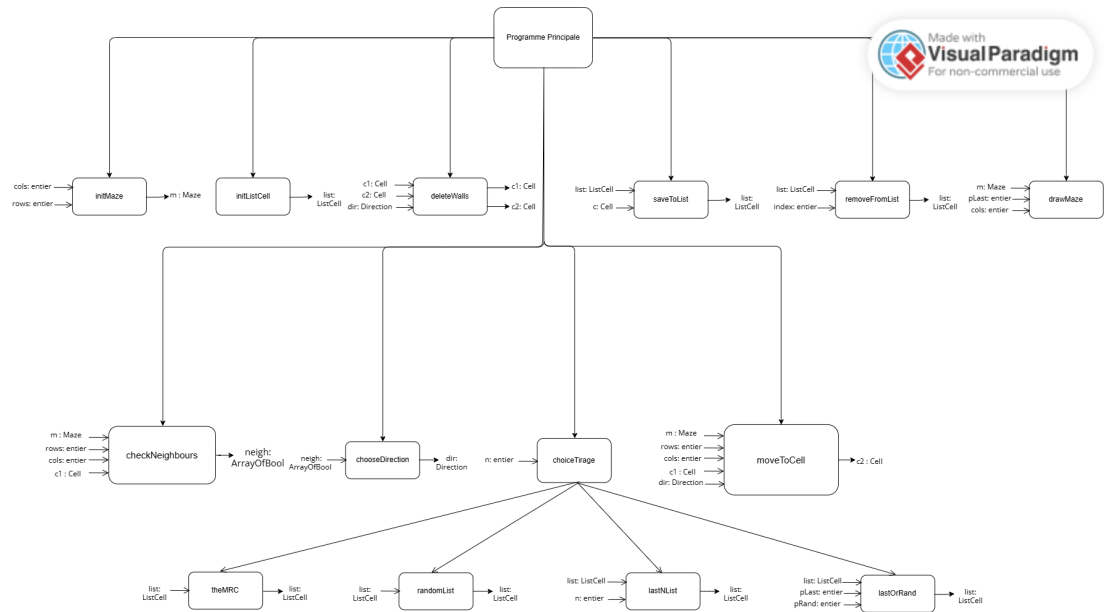


FIGURE 2.1 – Analyse Descendant

Chapitre 3

Implémentation

Nous avons choisi d'utiliser le langage C pour ce projet, car il permet une gestion fine de la mémoire et favorise la compréhension des structures de données. De plus, il correspond parfaitement aux objectifs pédagogiques, notamment la manipulation des tableaux et des pointeurs.

Le programme est organisé de manière modulaire pour séparer les responsabilités :

- **type.h** définit toutes les structures et énumérations nécessaires à la représentation du labyrinthe.
- **fonctions.c** / **fonctions.h** regroupent les fonctions utilitaires liées à la génération du labyrinthe (initialisation, affichage, gestion des murs, choix de direction, etc.).
- **growingTree.c** contient la fonction principal **main**, responsable des interactions avec l'utilisateur et du déroulement global de l'algorithme.
- **game_ncurses.c** / **game_ncurses.h** ajoutent un mode jeu interactif utilisant la bibliothèque *ncurses*.

Cette organisation rend le code plus clair, plus réutilisable et plus facile à tester.

Le labyrinthe est modélisé sous forme d'une matrice de cellules *Maze* de taille variable définie par l'utilisateur.

Chaque cellule *Cell* contient :

- ses coordonnées (x,y) dans la grille
- un état *visited* pour savoir si elle a déjà été explorée
- un tableau *walls[4]* représentant les murs.

Les cellules actives sont stockées dans une liste dynamique *ListCell* qui joue

un rôle central dans l'algorithme Growing Tree. Cette liste contient les cellules en cours de traitement et permet un accès flexible pour les différentes méthodes de tirage.

Les fonctions principales utilisées sont :

- *checkNeighbours()* pour repérer les voisins disponibles,
- *chooseDirection()* pour choisir un voisin aléatoire
- *deleteWall()* pour ouvrir un passage entre deux cellules
- *saveToList()* et *removeFromList()* pour gérer la liste active.

Nous avons implémenté quatre stratégies différentes pour le choix de la cellule active :

1. **theMRC** : on reprend toujours la dernière cellule ajoutée.
2. **randomList** : on choisit une cellule au hasard dans la liste.
3. **lastNList** : on tire au hasard parmi les N dernières cellules ajoutées.
4. **lastORrand** : on combine les deux approches selon des probabilités $pLast$ et $pRand$.

Deux modes d'affichage ont été intégrés :

- **Affichage ASCII** : la fonction *drawMaze()* affiche le labyrinthe directement dans la console. Les murs sont dessinés avec les caractères +, | et —, tandis que les espaces représentent les passages.
- Mode jeu avec ncurses : ce mode interactif permet à l'utilisateur de se déplacer dans le labyrinthe avec les touches fléchées pour atteindre la sortie. Le joueur est représenté par la lettre D et la sortie est marquée par G. Un mode marquage peut être activé pour laisser des traces X sur les cellules visitées.

Chapitre 4

Jeux d'essais et résultats

4.1 Méthode theMRC (Most Recent Cell)

Le labyrinthe généré présente des couloirs très fins et profonds, avec de nombreuses impasses. La structure est fortement ramifiée et la solution est longue et sinueuse. Le facteur rivière est bas, car les chemins sont étroits et peu connectés entre eux. Ce type de labyrinthe est intéressant pour les utilisateurs qui recherchent un défi plus difficile.

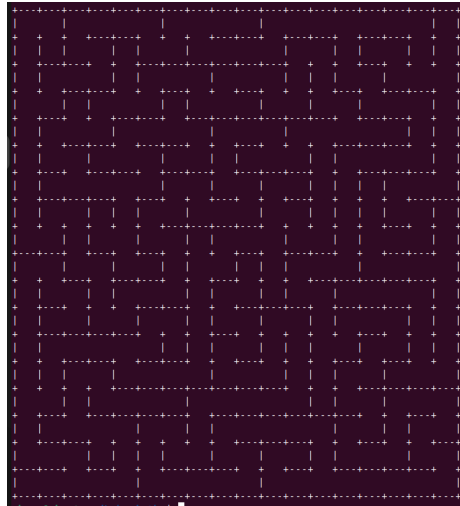


FIGURE 4.1 – theMRC

4.2 Méthode randomList

Le résultat est un labyrinthe désordonné et irrégulier, où les chemins se croisent souvent sans motif particulier. Le facteur rivière est moyen, car certaines zones sont denses alors que d'autres restent ouvertes. Cette méthode génère des labyrinthes à forme imprévisible et à solution variable, souvent plus difficiles. Visuellement, le labyrinthe paraît plus chaotique que celui obtenu avec MRC

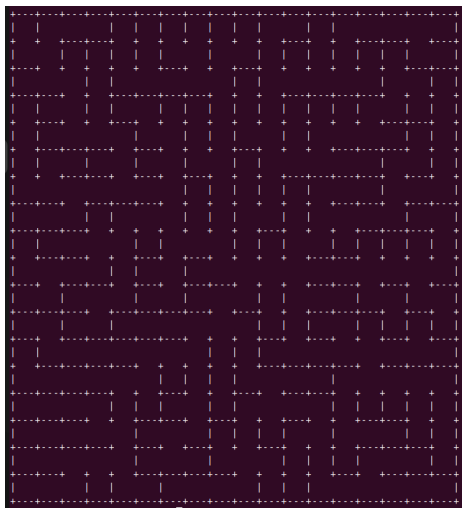


FIGURE 4.2 – random

4.3 Méthode lastNList

Cette variante permet de trouver un compromis entre les deux comportements précédents : le labyrinthe garde une certaine continuité comme *theMRC*, tout en intégrant un peu de hasard. Plus N est grand, plus la structure devient aléatoire. Le résultat montre un bon équilibre entre linéarité et diversité des chemins.

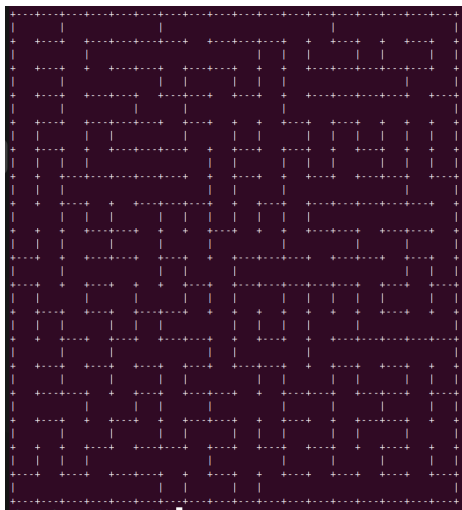


FIGURE 4.3 – lastNList

4.4 Méthode lastOrRand

Le labyrinthe obtenu est équilibré et fluide, avec des zones de continuité entrecoupées d'ouvertures aléatoires. Le facteur rivière est élevé, car les chemins sont plus larges et mieux connectés, mais la solution est plus courte et direct. Visuellement, le labyrinthe semble naturel et agréable à explorer.

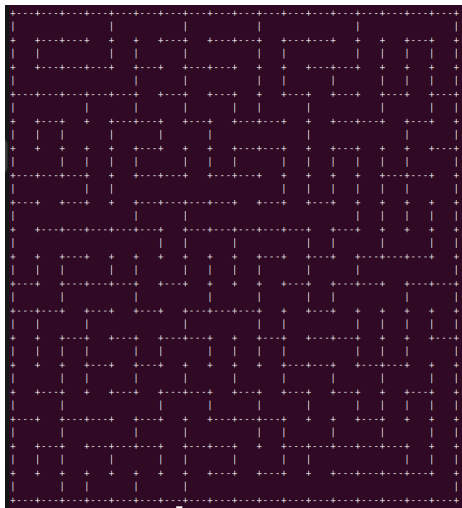


FIGURE 4.4 – lastOrRand

4.5 Comparaison

Méthode	Facteur rivière	Longueur de la solution	Structure générale	Style visuel
theMRC	Bas	Longue et sinueuse	Très ramifiée	Structuré et profond
randomList	Moyen	Variable	Densément connectée	Chaotique
lastNList	Bas	Longue et sinueuse	Dense et ramifiée	Imprévisible
lastOrRand	Élevé	Courte et directe	Fluide et équilibrée	Naturel et harmonieux

TABLE 4.1 – Comparaison des méthodes de génération de labyrinthes

4.6 Mode interactif

En plus de la génération du labyrinthe, nous avons intégré un mode jeu interactif utilisant la bibliothèque *ncurses*.

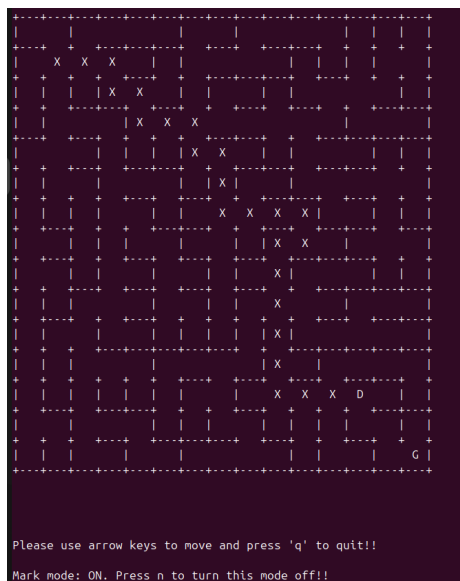


FIGURE 4.5 – Jouer avec *ncurses*

Chapitre 5

Synthèse critique

La réalisation de ce projet nous a permis de mettre en pratique des notions d’algorithmique et de programmation vues en cours, tout en développant de nouvelles compétences techniques. Le travail en binôme nous a aussi aidés à mieux comprendre l’importance de la répartition des tâches et de la communication dans un projet informatique.

La principale difficulté que nous avons rencontrée concernait le langage C, que nous n’avions jamais utilisé auparavant. Le passage en pseudocode vers une implémentation concrète en C s’est révélé compliqué au début, notamment à cause de certaines particularités du langage. Contrairement à d’autres langages que nous connaissions, le C ne gère pas directement les tableaux dynamiques, ce qui nous a obligés à apprendre à utiliser les pointeurs et la mémoire dynamique. Au départ, la manipulation des pointeurs n’était pas intuitive : nous avons rencontré beaucoup de erreurs de segmentation, de gestion des adresses mémoire, et la compréhension nous ont demandé beaucoup de temps et d’essais. Cependant, à force de pratique, nous avons aussi réussi à maîtriser ces aspects et à rendre notre code plus propre et plus efficace.

Ce projet nous a apporté une réelle progression dans la compréhension du langage C et de la gestion de la mémoire. Au delà de la technique, ce travail nous a appris la rigueur, la patience et la méthode nécessaire pour transformer un algorithme théorique en un programme fonctionnel. Si nous avions plus de temps et devons poursuivre ce projet, plusieurs améliorations seraient envisageables comme ajouter une fonction de résolution automatique du labyrinthe ou optimiser l’affichage graphique pour le rendre plus lisible et coloré.

Chapitre 6

Conclusion

Ce projet nous a permis de découvrir concrètement le processus complet de développement d'un programme, depuis la compréhension d'un algorithme jusqu'à sa mise en oeuvre effective en langage C. Nous avons pu observer comment un algorithme peut produire des résultats très variés selon les choix d'implémentation et les paramètres utilisés. Malgré les difficultés initiales liées à la manipulation des pointeurs et à la gestion de la mémoire, ce travail nous a beaucoup appris sur la logique du C et la programmation modulaire.

En conclusion, cette expérience a été riche sur le plan technique et pédagogique : elle nous a permis d'acquérir des compétences solides en algorithmique et en programmation bas niveau, tout en développant notre autonomie et notre capacité à apprendre nouvelles choses.